

Negotiated Local Plan Repair for Multi-Robot Warehouse Pathfinding

Sreenitya Thatikunta Arpita Deshmukh Prajna Agrawal
B23CS1072 B23CM1007 B23CS1054

1 Problem and environment

n robots move on a 4-connected grid. In each step a robot moves to a neighbouring free cell or waits. Vertex conflicts (two robots in one cell) and edge conflicts (swaps) are forbidden. Each robot has 3 pickup→delivery tasks and then returns to its own dock. The map is an 18×33 Kiva-style warehouse: 5×5 shelf blocks of 2×5 cells separated by one-cell aisles. It has 344 free cells, 150 pickup cells, 14 stations and 58 dead-end docks.

There are three kinds of disruption: **breakdowns** (5–20 steps, or permanent; a permanently broken robot’s tasks are auctioned to robots in range), **blockages** (10–40 steps, or permanent) and **emergency tasks** (a new pickup–delivery task that goes first, with a deadline of the ideal delivery time + 3 steps). Events are generated from the seed, so all strategies face the same schedule. The *dynamic obstacle density* ρ is the number of blockages divided by the number of free cells. By default a filter keeps every instance solvable (Section 4 turns it off). Robots within Manhattan distance $R = 6$ exchange messages (ALERT, REQUEST, PROPOSE, COMMIT, ...), and every message is counted.

2 Method

Initial plan. Prioritized planning: robots are planned one at a time (longest task chain first) with ST-A* over states $(x, y, \text{goal index}, t)$. Each path is written into a space-time reservation table (vertex, edge and parking reservations), and later robots plan around it.

Local repair. When a disruption hits, the robots whose plans it invalidates are repaired one at a time, trying the options that change the fewest plans first (Figure 1). For robot a , let ℓ^* be its robot-free path and $d(p) = |p| - |\ell^*|$ the delay of a path p . A coalition S of neighbours that replan jointly with a (with a planned first) is scored by

$$\text{score}(S) = d(p_a) + \sum_{m \in S} \Delta_m + \lambda|S| + \text{DL}(p_a), \quad (1)$$

where Δ_m is member m ’s added completion time, $\lambda = 3$ is the price of changing one more robot’s plan, and DL adds a penalty of 10 if an emergency deadline is missed. A coalition is accepted only if it scores better than going solo.

- **Tier 1, solo:** replan a alone with all other plans fixed. Accept if the delay is at most $\delta = 4$ (0 for emergency robots).
- **Tiers 2–3, negotiation** within R , then $2R$. Candidates are the movable robots that block ℓ^* . Coalitions of size 1, 2, 3 are tried in order, and the search stops at the first size that beats going solo. A member that cannot dodge may recruit its own blockers (a *chain*, at most 2 hops and 6 robots), scored as one tree.
- **Tier 3b, local group replan:** for urgent robots (emergency, or no solo path), prioritized ST-A* for the at most 8 robots within $2R$ that cross a ’s path.
- **Tier 4, hold:** otherwise a takes its solo detour or, if none exists, waits in place and retries every step. After 10 steps on hold its search radius doubles (expanding ring).

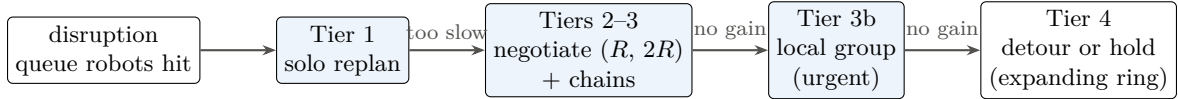


Figure 1: Repair tiers. A robot moves to the next tier only if the current one cannot beat going solo, so the number of robots touched grows only when needed.

We compare **local** (the method above), **local-flat** (our first version, without chains, Tier 3b or deadlines), **solo** (Tier 1 only, an ablation) and **full**, which re-runs prioritized ST-A* for every robot after every disruption. **full** is exactly what the assignment forbids, and serves as a reference.

3 Experiments and results

Each setting was run for 10 seeds with every strategy on identical instances. Defaults are 3% density (20% of blockages permanent), 2 breakdowns and 2 emergencies per run. We varied the number of robots (5–50), the density (0–15% with 20 robots), the alter penalty ($\lambda = 0$ –12), and ran single-disruption runs of each type. **Altered** counts the robots whose future plan changed because of a disruption (including cascades). Every executed step is checked for collisions. All 1,700 runs had zero collisions, and all completed every task except one **solo** run (Section 4).

Table 1: Number of robots (means over 10 seeds, 12 disruptions per run). SoC = sum of task completion times.

n	strategy	SoC	altered /dis.	emerg. on time	ms /dis.	n	strategy	SoC	altered /dis.	emerg. on time	ms /dis.
20	local	2612	2.06	65%	126	50	local	6958	5.88	35%	2270
	local-flat	2608	2.11	45%	81		local-flat	6974	5.75	15%	1102
	solo	2622	1.93	30%	12		solo	7089	4.62	15%	67
	full	2611	9.22	100%	41		full	7201	39.33	100%	590

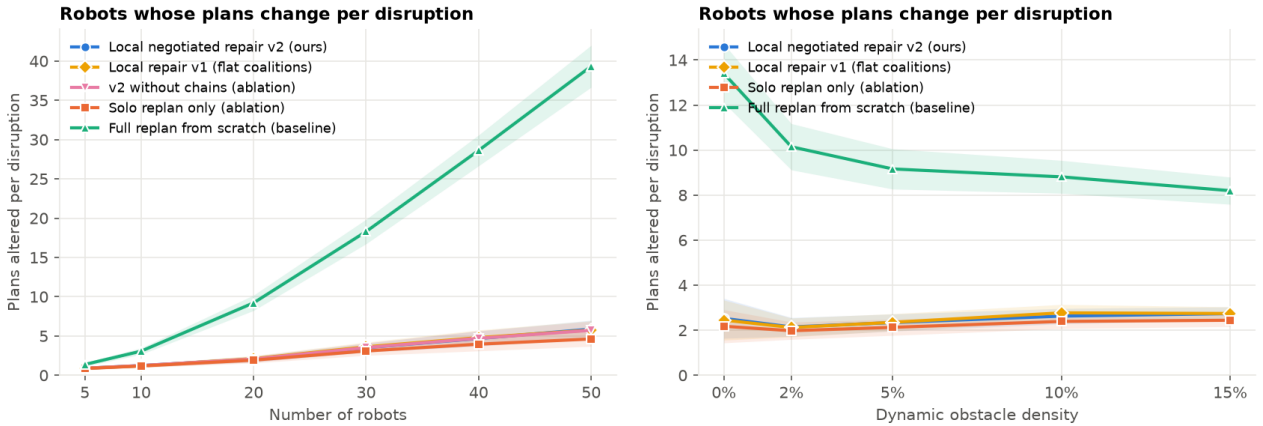


Figure 2: Plans changed per disruption vs. number of robots (left) and dynamic obstacle density (right, 20 robots). Bands are 95% confidence intervals.

Plans changed. Local repair changes about one robot beyond those a disruption hits directly. Full replanning changes 9–39 robots per disruption, nearly the whole fleet, and is not faster: at 50 robots its SoC is *worse* (7201 vs 6958), because replanning everyone reshuffles priorities and delays robots that were not affected. In single-disruption runs, a temporary breakdown changes 2.2 plans (exactly the robots hit), a temporary blockage 3.1, and a permanent breakdown 13.5 (mostly the robots hit directly plus the winners of its tasks), against 22–29 for full replanning.

Total time and scaling. **local** has the lowest or near-lowest SoC at every size. Negotiation matters more as the fleet grows: **solo** needs 131 more steps at 50 robots. Compared with **local-flat**, the final version lowers SoC by 7–19 steps at 30–50 robots and makes more emergencies on time (35%

vs 15% at 50 robots), changing about as many plans. The cost is about twice the repair time, which reaches 2.3s per disruption at 50 robots. From 0% to 15% density, the extra time grows from 143 to 503 steps while plans changed per disruption stay between 2.1 and 2.7. Raising λ from 0 to 12 trades a few extra steps for fewer altered robots; at 40 robots our default $\lambda = 3$ beats the first version at any λ on both measures.

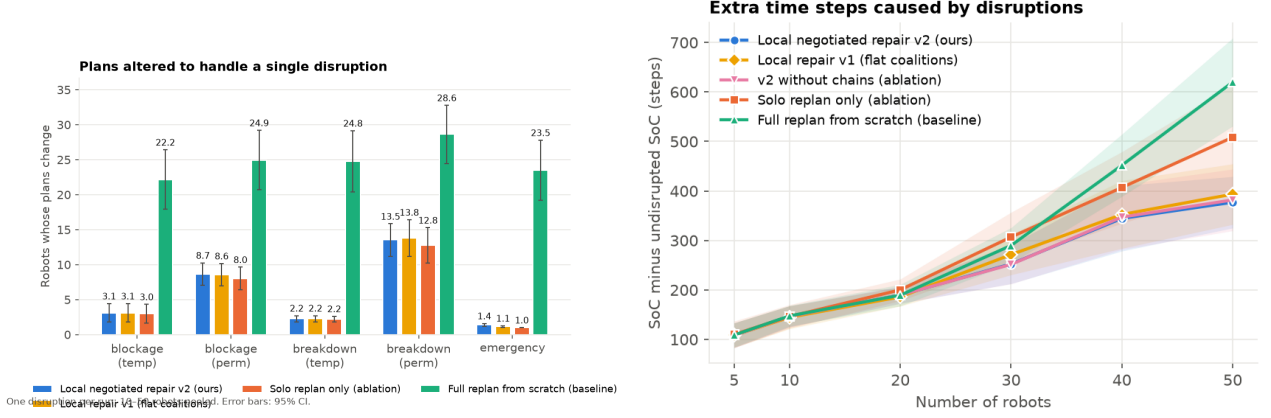


Figure 3: Left: plans changed to handle a single disruption, by type (10–50 robots pooled). Right: extra time steps caused by disruptions vs. number of robots.

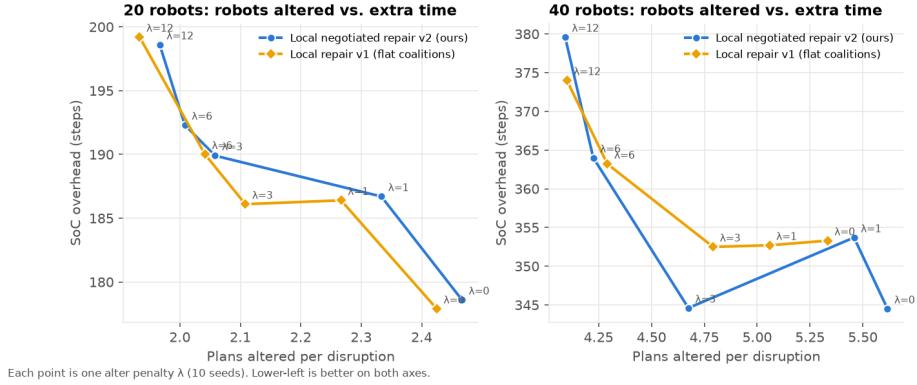


Figure 4: Plans changed vs. extra time as the alter penalty λ varies (ours vs. our first version).

4 Settings where agents fail

In the main sweeps robots rarely fail, partly because the generator filters out disruptions that make a task impossible. A **stress study** (760 runs, 10 seeds, all strategies) switches the filter off and pushes the environment further. A run *fails* if a task is undone when it ends (every robot done, or nothing moved for 150 steps). Each stuck robot is labelled *goal cut off* (its goal is unreachable, so the instance is unsolvable) or *deadlocked* (every goal reachable, but stuck).

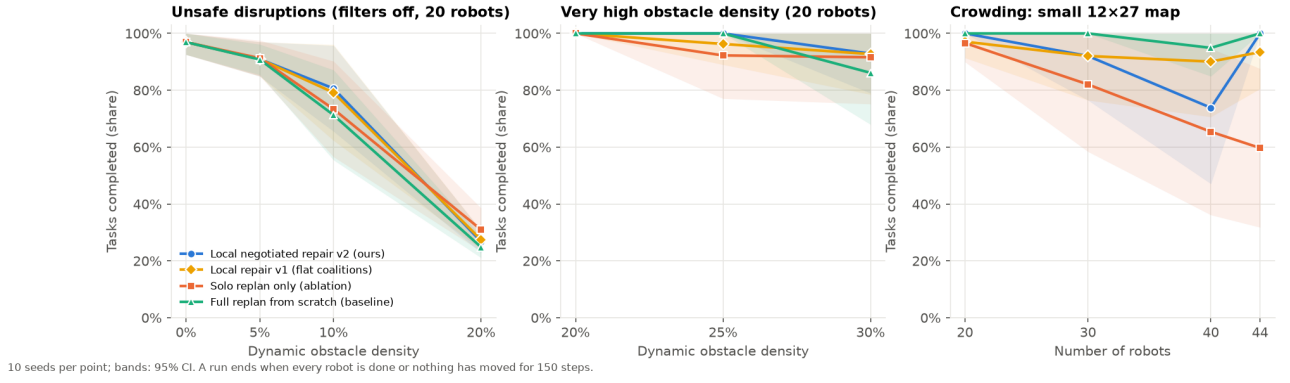


Figure 5: Share of tasks completed in the stress settings (10 seeds per point).

Table 2: Where agents fail: runs failed out of 10 and tasks completed; stuck robots per run under `local` (cut off / deadlocked).

setting	value	local (ours)			local-flat	full
		failed	completed	stuck: cut / dead	completed	completed
filter off, half of events permanent	$\rho = 0\%$	3/10	96.9%	0.5 / 0.2	96.9%	96.9%
	$\rho = 5\%$	7/10	91.0%	1.5 / 0.3	91.0%	90.8%
	$\rho = 10\%$	10/10	80.6%	2.3 / 3.1	79.2%	71.3%
	$\rho = 20\%$	10/10	26.8%	9.9 / 8.8	27.4%	24.8%
one perm. breakdown, filter off	20 robots	2/10	97.5%	0.4 / 0.2	97.5%	97.5%
	40 robots	4/10	96.7%	0.4 / 1.0	96.7%	96.7%
very dense, filter on	$\rho = 25\%$	0/10	100%	—	96.3%	100%
	$\rho = 30\%$	1/10	92.9%	0.0 / 2.0	92.7%	86.1%
crowded 12×27 map	30 robots	1/10	92.1%	0.0 / 2.9	92.1%	100%
	40 robots	3/10	73.9%	0.0 / 11.9	90.1%	94.9%
	44 robots	0/10	100%	—	93.4%	100%
comm. radius, 40 robots	$R = 1, 2, 3, 6$	0/10	100%	—	100%	100%

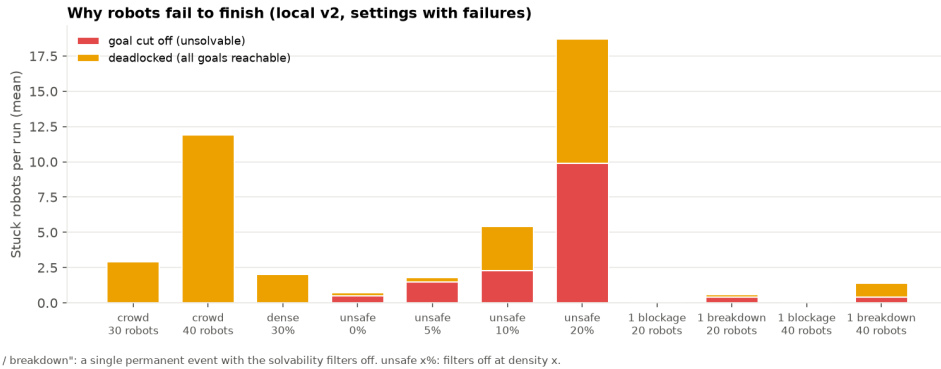


Figure 6: Why robots fail to finish under our method: goal cut off (red) or deadlocked although every goal is reachable (orange).

Goals cut off. With the filter off, a permanent blockage or a robot that breaks down for good can seal off a pickup cell, a station or an aisle segment (Figure 7). No repair can finish those tasks, so all strategies fail alike: completion drops from 97% to 27% as density rises to 20%. A single permanent breakdown already makes 2–4 of 10 runs fail, because a dead robot in a one-cell aisle is a wall.

Knock-on deadlock. A robot whose goal is cut off never gives up its task and holds its cell forever, which can block other robots' only route. At 10% density, 2.3 robots per run are cut off and another 3.1 are deadlocked behind them, so more than half of the failures come from the agents' reaction rather than the instance. Local repair suffers less than full replanning here (81% vs 71% completed).

Hold cascade on crowded maps (our main weakness). On a small map with 40 robots, 3 of 10 `local` runs fail, each with almost the whole fleet stuck. In one, a *temporary* breakdown blocks 6 robots, which hold. Every robot planned through a held cell then holds too, until all 40 robots wait on each other and only 1 of 122 tasks is delivered. Coalitions (at most 4 robots) and Tier 3b (at most 8) are too small to untangle this, while full replanning solves the same instance. The effect depends on the instance: at 44 robots no run failed.

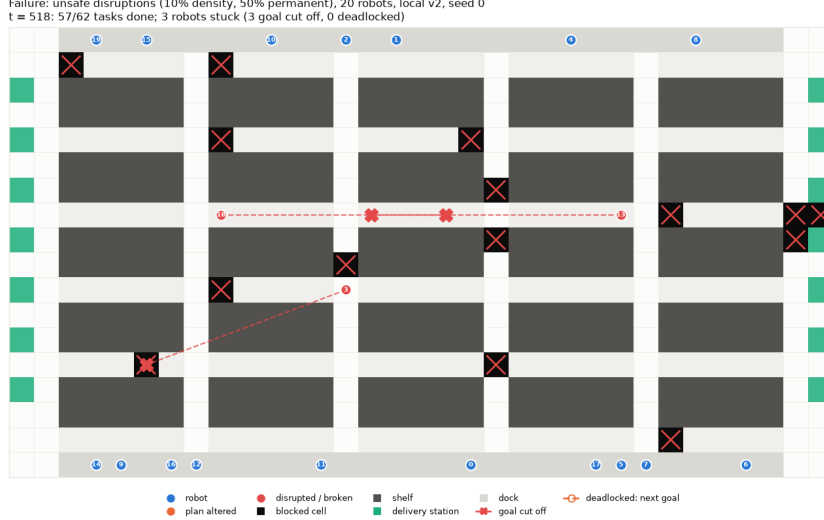


Figure 7: A failure from the demonstration: permanent blockages (black) cut off the goals of robots 3, 13 and 18, and 5 of 62 tasks are never done.

Other settings. At 30% density (all tasks solvable), 1 of 10 runs deadlocks. Without negotiation (`solo`), all 49 remaining robots gridlock after one permanent breakdown with 50 robots, and 5 of 10 runs fail on the crowded map with 44 robots; `local` solves both. Emergency deadlines are met in only 5–35% of cases in dense fleets, though the tasks are eventually delivered. The agents did *not* fail with a short communication radius ($R = 1$ costs only 1% more time, since holding robots widen their search), nor in any solvable instance on the main map with up to 50 robots and 20% density.

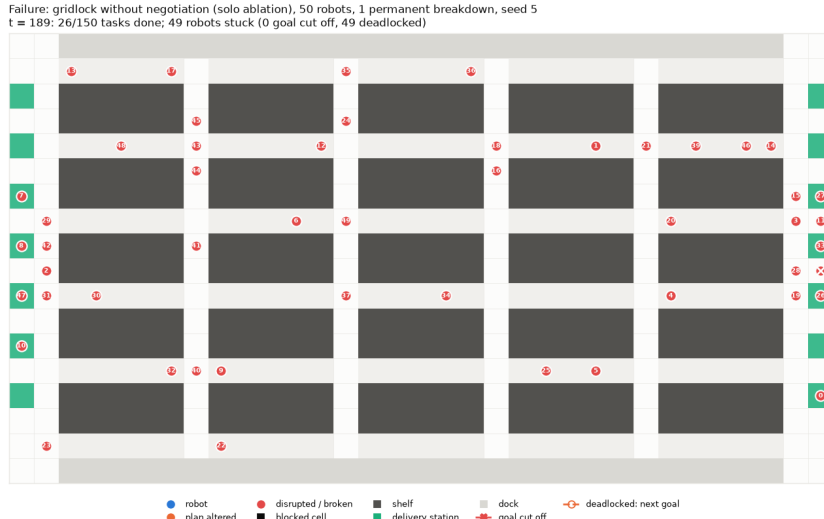


Figure 8: Gridlock without negotiation (`solo`): after one permanent breakdown all 49 remaining robots (red) wait on each other. Our method completes all 150 tasks on the same instance.

5 Graphical demonstration

Simulations are rendered with matplotlib: an MP4 video of a 20-robot run with mixed disruptions (Figure 9), an interactive replay in the browser, before/after snapshots of each disruption type (old

plan dashed, repaired plan solid), MP4 videos of the failure cases above, and a live replay window. For 8 steps after each disruption, the video highlights the robots whose plans changed and draws their repaired paths.

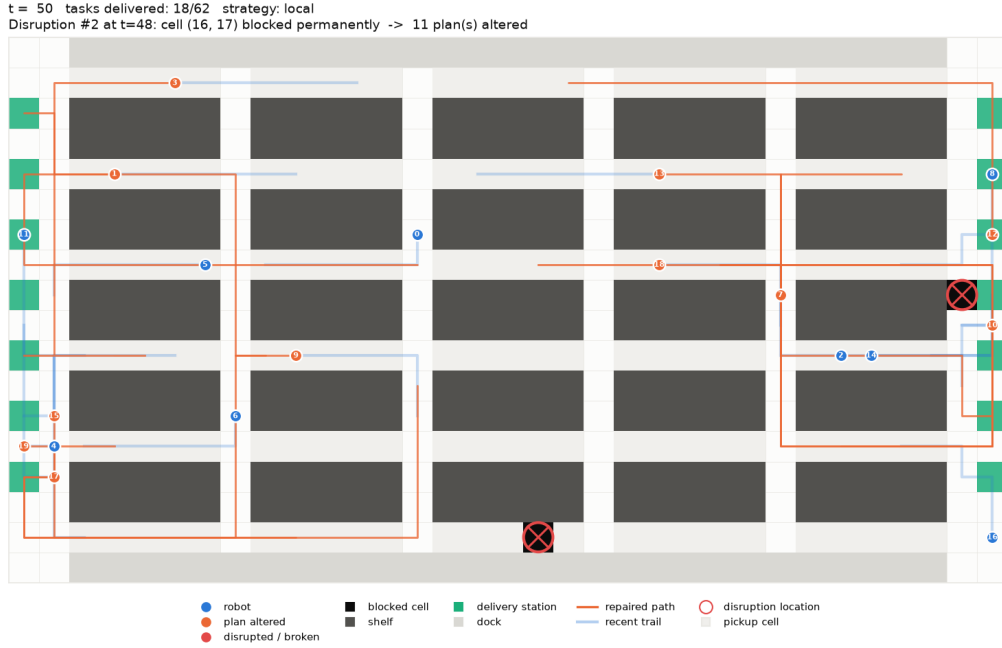


Figure 9: A frame from the demonstration video, two steps after a permanent blockage (red circle) that changed 11 plans. Orange robots have altered plans; orange lines are their repaired paths.

6 Limitations

- **Incomplete planner:** prioritized planning does not guarantee a solution when one exists, either for the initial plan or for joint replans during repair.
- **No task release:** a robot whose goal is cut off holds its task and its cell forever, which can deadlock other robots.
- **Hold cascades:** on crowded maps one local hold can spread to the whole fleet; an escalation to a larger regional replan is needed.
- **Soft emergency deadlines:** the deadline only changes which repair is preferred, so in dense fleets most deadlines are missed.
- **Repair time:** it reaches about 2 s per disruption at 50 robots in Python.
- **Idealized communication:** messages are instant and never lost.

Code, results and demos: <https://github.com/SreenityaThatikunta/warehouse-plan-repair>